

Ejercicio 1. Control de versiones con Git y GitHub

1. Sistema de control de versiones y el problema que resuelve

Un sistema de control de versiones (VCS) es una herramienta de software que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones específicas en el futuro. En el trabajo en equipo, resuelve el problema crítico de la concurrencia y la sobrescritura de código. Sin un VCS, si dos personas editan el mismo archivo al mismo tiempo, los cambios de uno aplastarán los del otro. Un VCS permite que múltiples desarrolladores trabajen en paralelo, fusionando sus contribuciones de manera ordenada y manteniendo un historial auditable de quién hizo qué y por qué.

2. Diferencia entre Git y GitHub

Git es el motor; GitHub es el servicio de alojamiento. Git es un sistema de control de versiones distribuido que se instala de forma local en tu computadora y funciona a través de la línea de comandos para gestionar el historial de tu código. GitHub, por otro lado, es una plataforma en la nube que aloja repositorios de Git y añade una interfaz gráfica y herramientas de colaboración avanzadas, como el control de acceso, foros de discusión y gestión de Pull Requests.

3. Conceptos clave con ejemplos

*** **Repositorio:** Es el directorio donde se almacenan los archivos de tu proyecto junto con todo su historial de cambios. ***Ejemplo:*** La carpeta local ``practica1-bd`` que contiene tus scripts SQL y documentos PDF.

Confirmación (commit): Es una "fotografía" o punto de control del estado de tus archivos en un momento dado. ***Ejemplo:*** Ejecutar ``git commit -m "Agrega esquema inicial de la base de datos"``.

Rama (branch): Es una línea independiente de desarrollo que permite trabajar en una nueva funcionalidad sin afectar el código estable. ***Ejemplo:*** Crear una rama ``feature-consultas`` para probar unos scripts antes de integrarlos al proyecto principal.

Fusión (merge): Es el proceso de unir el historial y los cambios de dos ramas distintas en una sola. ***Ejemplo:*** Integrar los cambios de ``feature-consultas`` de vuelta a la rama principal (``main``).

Conflicto de fusión: Ocurre cuando Git no puede resolver automáticamente una fusión, generalmente porque la misma línea de un archivo fue modificada de forma distinta en ambas ramas. *Ejemplo:* Dos compañeros modificaron la línea 15 del `README.md` con descripciones diferentes y tú debes elegir manualmente cuál conservar.

Pull Request: Es una solicitud en plataformas como GitHub para que los administradores de un repositorio revisen los cambios de tu rama antes de fusionarla con la rama principal. *Ejemplo:* Terminar tu parte de la práctica y abrir un Pull Request para que tu equipo valide que el código cumple los requisitos.

Archivo .gitignore: Es un archivo de texto que indica a Git qué archivos o directorios debe ignorar y no rastrear. *Ejemplo:* Un archivo que oculta archivos compilados (`.exe`, `.o`) de entornos como Dev-C++ o directorios de datos.

Archivo README: Es el documento de presentación de un repositorio que explica de qué trata el proyecto, cómo instalarlo y cómo usarlo. *Ejemplo:* Un `README.md` que incluye tu nombre, carrera y un índice del repositorio.

4. Flujo de trabajo basado en ramas y revisión por pares

Un flujo de trabajo basado en ramas consiste en mantener una rama principal que siempre contiene código funcional y estable. Cualquier nueva característica se desarrolla en una rama paralela efímera. La revisión por pares se realiza antes de fusionar (usualmente durante el Pull Request); esto es fundamental porque garantiza la calidad del código, unifica el estilo de programación, permite detectar errores de lógica que el autor original pasó por alto y distribuye el conocimiento del sistema entre todo el equipo.